JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang

Class Class<T>

java.lang.Object
    java.lang.Class<T>

Type Parameters:

`T - the type of the class modeled by this Class object. For example, the type of String.class is Class<String>. Use Class<?> if the class being modeled is unknown.`

All Implemented Interfaces:

`Serializable, AnnotatedElement, GenericDeclaration, Type`

```
public final class Class<T>
extends Object
implements Serializable, GenericDeclaration, Type, AnnotatedElement
```

Instances of the class `Class` represent classes and interfaces in a running Java application. An enum is a kind of class and an annotation is a kind of interface. Every array also belongs to a class that is reflected as a `Class` object that is shared by all arrays with the same element type and number of dimensions. The primitive Java types (`boolean`, `byte`, `char`, `short`, `int`, `long`, `float`, and `double`), and the keyword `void` are also represented as `Class` objects.

`Class` has no public constructor. Instead `Class` objects are constructed automatically by the Java Virtual Machine as classes are loaded and by calls to the `defineClass` method in the class loader.

The following example uses a `Class` object to print the class name of an object:

```
void printClassName(Object obj) {
    System.out.println("The class of " + obj +
                       " is " + obj.getClass().getName());
}
```

It is also possible to get the `Class` object for a named type (or for void) using a class literal. See Section 15.8.2 of *The Java™ Language Specification*. For example:

```
System.out.println("The name of class Foo is:
"+Foo.class.getName());
```

Since:

JDK1.0

See Also:

`ClassLoader.defineClass(byte[], int, int)`, `Serialized Form`

Method Summary

Concrete Methods

Modifier and Type	Method
<U> **Class**<? extends U>	**asSubclass**(**Class**<U> clazz)
T	**cast**(**Object** obj)
boolean	**desiredAssertionStatus**()
static **Class**<?>	**forName**(**String** className)
static **Class**<?>	**forName**(**String** name, boolean initialize, **ClassLoader** loader)

AnnotatedType[]	getAnnotatedInterfaces()
AnnotatedType	getAnnotatedSuperclass()
<A extends Annotation> A	getAnnotation(Class<A> annotationClass)
Annotation[]	getAnnotations()
<A extends Annotation> A[]	getAnnotationsByType(Class<A> annotationClass)
String	getCanonicalName()
Class<?>[]	getClasses()

ClassLoader	**getClassLoader**()
Class<?>	**getComponentType**()
Constructor<**T**>	**getConstructor**(**Class**<?>... parameterTypes)
Constructor<?>[]	**getConstructors**()
<A extends **Annotation**> A	**getDeclaredAnnotation**(**Class**<A> annotationClass)
Annotation[]	**getDeclaredAnnotations**()
<A extends **Annotation**> A[]	**getDeclaredAnnotationsByType**(**Class**<A> annotationClass)
Class<?>[]	**getDeclaredClasses**()

Constructor<**T**>	**getDeclaredConstructor**(**Class**<?>... parameterTypes)
Constructor<?>[]	**getDeclaredConstructors**()
Field	**getDeclaredField**(**String** name)
Field[]	**getDeclaredFields**()
Method	**getDeclaredMethod**(**String** name, **Class**<?>... parameterTypes)

`Method[]`	`getDeclaredMethods()`
`Class<?>`	`getDeclaringClass()`
`Class<?>`	`getEnclosingClass()`
`Constructor<?>`	`getEnclosingConstructor()`
`Method`	`getEnclosingMethod()`

T[]	getEnumConstants()
Field	getField(String name)
Field[]	getFields()
Type[]	getGenericInterfaces()
Type	getGenericSuperclass()
Class<?>[]	getInterfaces()

Method	**getMethod**(**String** name, **Class**<?>... parameterTypes)
Method[]	**getMethods**()
int	**getModifiers**()
String	**getName**()
Package	**getPackage**()
ProtectionDomain	**getProtectionDomain**()
URL	**getResource**(**String** name)

InputStream	getResourceAsStream(String name)
Object[]	getSigners()
String	getSimpleName()
Class<? super T>	getSuperclass()
String	getTypeName()
TypeVariable<Class<T>>[]	getTypeParameters()
boolean	isAnnotation()
boolean	isAnnotationPresent(Class<? extends Annotation> annotationClass)

boolean	**isAnonymousClass**()
boolean	**isArray**()
boolean	**isAssignableFrom**(**Class**<?> cls)
boolean	**isEnum**()
boolean	**isInstance**(**Object** obj)
boolean	**isInterface**()
boolean	**isLocalClass**()
boolean	**isMemberClass**()

boolean	**isPrimitive**()
boolean	**isSynthetic**()
T	**newInstance**()
String	**toGenericString**()
String	**toString**()

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Method Detail

toString

```
public String toString()
```

Converts the object to a string. The string representation is the string "class" or "interface", followed by a space, and then by the fully qualified name of the class in the format returned by `getName`. If this `Class` object represents a primitive type, this method returns the name of the primitive type. If this `Class` object represents void this method returns "void".

Overrides:

`toString` in class `Object`

Returns:

a string representation of this class object.

toGenericString

```
public String toGenericString()
```

Returns a string describing this `Class`, including information about modifiers and type parameters. The string is formatted as a list of type modifiers, if any, followed by the kind of type (empty string for primitive types and `class`, `enum`, `interface`, or `@interface`, as appropriate), followed by the type's name, followed by an angle-bracketed comma-separated list of the type's type parameters, if any. A space is used to separate modifiers from one another and to separate any modifiers from the kind of type. The modifiers occur in canonical order. If there are no type parameters, the type parameter list is elided.

Note that since information about the runtime representation of a type is being generated, modifiers not present on the originating source code or illegal on the originating source code may be present.

Returns:

a string describing this Class, including information about modifiers and type parameters

Since:

1.8

forName

```
public static Class<?> forName(String className)
                        throws ClassNotFoundException
```

Returns the `Class` object associated with the class or interface with the given string name. Invoking this method is equivalent to:

```
Class.forName(className, true, currentLoader)
```

where `currentLoader` denotes the defining class loader of the current class.

For example, the following code fragment returns the runtime `Class` descriptor for the class named `java.lang.Thread`:

```
Class t = Class.forName("java.lang.Thread")
```

A call to `forName("X")` causes the class named `X` to be initialized.

Parameters:

`className - the fully qualified name of the desired class.`

Returns:

`the Class object for the class with the specified name.`

Throws:

`LinkageError - if the linkage fails`

`ExceptionInInitializerError - if the initialization provoked by this method fails`

`ClassNotFoundException - if the class cannot be located`

forName

```
public static Class<?> forName(String name,
                               boolean initialize,
                               ClassLoader loader)
                        throws ClassNotFoundException
```

Returns the `Class` object associated with the class or interface with the given string name, using the given class loader. Given the fully qualified name for a class or interface (in the same format returned by `getName`) this method attempts to locate, load, and link the class or interface. The specified class loader is used to load the class or interface. If the parameter `loader` is null, the class is loaded through the bootstrap class loader. The class is initialized only if the `initialize` parameter is `true` and if it has not been initialized earlier.

If `name` denotes a primitive type or void, an attempt will be made to locate a user-defined class in the unnamed package whose name is `name`. Therefore, this method cannot be used to obtain any of the `Class` objects representing primitive types or void.

If `name` denotes an array class, the component type of the array class is loaded but not initialized.

For example, in an instance method the expression:

```
Class.forName("Foo")
```

is equivalent to:

```
Class.forName("Foo", true,
this.getClass().getClassLoader())
```

Note that this method throws errors related to loading, linking or initializing as specified in Sections 12.2, 12.3 and 12.4 of *The Java Language Specification*. Note that this method does not check whether the requested class is accessible to its caller.

If the `loader` is `null`, and a security manager is present, and the caller's class loader is not null, then this method calls the security manager's `checkPermission` method with a `RuntimePermission("getClassLoader")` permission to ensure it's ok to access the bootstrap class loader.

Parameters:

`name` - fully qualified name of the desired class

`initialize` - if true the class will be initialized. See Section 12.4 of *The Java Language Specification*.

`loader` - class loader from which the class must be loaded

Returns:

class object representing the desired class

Throws:

`LinkageError` - if the linkage fails

`ExceptionInInitializerError` - if the initialization provoked by this method fails

`ClassNotFoundException` - if the class cannot be located by the specified class loader

Since:

1.2

See Also:

`forName(String)`, `ClassLoader`

newInstance

```
public T newInstance()
             throws InstantiationException,
                    IllegalAccessException
```

Creates a new instance of the class represented by this `Class` object. The class is instantiated as if by a `new` expression with an empty argument list. The class is initialized if it has not already been initialized.

Note that this method propagates any exception thrown by the nullary constructor, including a checked exception. Use of this method effectively bypasses the compile-time exception checking that would otherwise be performed by the compiler. The `Constructor.newInstance` method avoids this problem by wrapping any exception thrown by the constructor in a (checked) `InvocationTargetException`.

Returns:

a newly allocated instance of the class represented by this object.

Throws:

`IllegalAccessException` - if the class or its nullary constructor is not accessible.

`InstantiationException` - if this Class represents an abstract class, an interface, an array class, a primitive type, or void; or if the class has no nullary constructor; or if the instantiation fails for some other reason.

`ExceptionInInitializerError` - if the initialization provoked by this method fails.

`SecurityException` - If a security manager, *s*, is present and the caller's class loader is not the same as or an ancestor of the class loader for the current class and invocation of `s.checkPackageAccess()` denies access to the package of this class.

isInstance

```
public boolean isInstance(Object obj)
```

Determines if the specified `Object` is assignment-compatible with the object represented by this `Class`. This method is the dynamic equivalent of the Java language `instanceof` operator. The method returns `true` if the specified `Object` argument is non-null and can be cast to the reference type represented by this `Class` object without raising a `ClassCastException.` It returns `false` otherwise.

Specifically, if this `Class` object represents a declared class, this method returns `true` if the specified `Object` argument is an instance of the represented class (or of any of its subclasses); it returns `false` otherwise. If this `Class` object represents an array class, this method returns `true` if the specified `Object` argument can be converted to an object of the array class by an identity conversion or by a widening reference conversion; it returns `false` otherwise. If this `Class` object represents an interface, this method

returns `true` if the class or any superclass of the specified `Object` argument implements this interface; it returns `false` otherwise. If this `Class` object represents a primitive type, this method returns `false`.

Parameters:

`obj - the object to check`

Returns:

`true if obj is an instance of this class`

Since:

`JDK1.1`

isAssignableFrom

```
public boolean isAssignableFrom(Class<?> cls)
```

Determines if the class or interface represented by this `Class` object is either the same as, or is a superclass or superinterface of, the class or interface represented by the specified `Class` parameter. It returns `true` if so; otherwise it returns `false`. If this `Class` object represents a primitive type, this method returns `true` if the specified `Class` parameter is exactly this `Class` object; otherwise it returns `false`.

Specifically, this method tests whether the type represented by the specified `Class` parameter can be converted to the type represented by this `Class` object via an identity conversion or via a widening reference conversion. See *The Java Language Specification*, sections 5.1.1 and 5.1.4 , for details.

Parameters:

`cls - the Class object to be checked`

Returns:

`the boolean value indicating whether objects of the type cls can be assigned to objects of this class`

Throws:

`NullPointerException - if the specified Class parameter is null.`

Since:

`JDK1.1`

isInterface

```
public boolean isInterface()
```

Determines if the specified `Class` object represents an interface type.

Returns:

`true if this object represents an interface; false otherwise.`

isArray

```
public boolean isArray()
```

Determines if this `Class` object represents an array class.

Returns:

`true if this object represents an array class; false otherwise.`

Since:

`JDK1.1`

isPrimitive

```
public boolean isPrimitive()
```

Determines if the specified `Class` object represents a primitive type.

There are nine predefined `Class` objects to represent the eight primitive types and void. These are created by the Java Virtual Machine, and have the same names as the primitive types that they represent, namely `boolean`, `byte`, `char`, `short`, `int`, `long`, `float`, and `double`.

These objects may only be accessed via the following public static final variables, and are the only `Class` objects for which this method returns `true`.

Returns:

`true if and only if this class represents a primitive type`

Since:

`JDK1.1`

See Also:

`Boolean.TYPE`, `Character.TYPE`, `Byte.TYPE`, `Short.TYPE`, `Integer.TYPE`, `Long.TYPE`, `Float.TYPE`, `Double.TYPE`, `Void.TYPE`

isAnnotation

```
public boolean isAnnotation()
```

Returns true if this `Class` object represents an annotation type. Note that if this method returns true, `isInterface()` would also return true, as all annotation types are also interfaces.

Returns:

`true if this class object represents an annotation type; false otherwise`

Since:

`1.5`

isSynthetic

```
public boolean isSynthetic()
```

Returns `true` if this class is a synthetic class; returns `false` otherwise.

Returns:

`true if and only if this class is a synthetic class as defined by the Java Language Specification.`

Since:

`1.5`

See *The Java™ Language Specification*:

`13.1 The Form of a Binary`

getName

```
public String getName()
```

Returns the name of the entity (class, interface, array class, primitive type, or void) represented by this `Class` object, as a `String`.

If this class object represents a reference type that is not an array type then the binary name of the class is returned, as specified by *The Java™ Language Specification*.

If this class object represents a primitive type or void, then the name returned is a `String` equal to the Java language keyword corresponding to the primitive type or void.

If this class object represents a class of arrays, then the internal form of the name consists of the name of the element type preceded by one or more '['

characters representing the depth of the array nesting. The encoding of element type names is as follows:

Element Type	Encoding
boolean	Z
byte	B
char	C
class or interface	L*classname*;
double	D
float	F
int	I
long	J
short	S

The class or interface name *classname* is the binary name of the class specified above.

Examples:

```
String.class.getName()
    returns "java.lang.String"
byte.class.getName()
    returns "byte"
(new Object[3]).getClass().getName()
    returns "[Ljava.lang.Object;"
(new int[3][4][5][6][7][8][9]).getClass().getName()
    returns "[[[[[[[I"
```

Returns:

the name of the class or interface represented by this object.

getClassLoader

```
public ClassLoader getClassLoader()
```

Returns the class loader for the class. Some implementations may use null to represent the bootstrap class loader. This method will return null in such implementations if this class was loaded by the bootstrap class loader.

If a security manager is present, and the caller's class loader is not null and the caller's class loader is not the same as or an ancestor of the class loader for the class whose class loader is requested, then this method calls the security manager's `checkPermission` method with a `RuntimePermission("getClassLoader")` permission to ensure it's ok to access the class loader for the class.

If this object represents a primitive type or void, null is returned.

Returns:

the class loader that loaded the class or interface represented by this object.

Throws:

SecurityException - if a security manager exists and its checkPermission method denies access to the class loader for the class.

See Also:

ClassLoader, SecurityManager.checkPermission(java.security.Permission), RuntimePermission

getTypeParameters

```
public TypeVariable<Class<T>>[] getTypeParameters()
```

Returns an array of TypeVariable objects that represent the type variables declared by the generic declaration represented by this GenericDeclaration object, in declaration order. Returns an array of length 0 if the underlying generic declaration declares no type variables.

Specified by:

getTypeParameters in interface GenericDeclaration

Returns:

an array of TypeVariable objects that represent the type variables declared by this generic declaration

Throws:

GenericSignatureFormatError - if the generic signature of this generic declaration does not conform to the format specified in *The Java™ Virtual Machine Specification*

Since:

1.5

getSuperclass

```
public Class<? super T> getSuperclass()
```

Returns the Class representing the superclass of the entity (class, interface, primitive type or void) represented by this Class. If this Class represents either the Object class, an interface, a primitive type, or void, then null is returned. If this object represents an array class then the Class object representing the Object class is returned.

Returns:

the superclass of the class represented by this object.

getGenericSuperclass

```
public Type getGenericSuperclass()
```

Returns the Type representing the direct superclass of the entity (class, interface, primitive type or void) represented by this Class.

If the superclass is a parameterized type, the Type object returned must accurately reflect the actual type parameters used in the source code. The parameterized type representing the superclass is created if it had not been created before. See the declaration of ParameterizedType for the semantics of the creation process for parameterized types. If this Class represents either the Object class, an interface, a primitive type, or void, then null is returned. If this object represents an array class then the Class object representing the Object class is returned.

Returns:

the superclass of the class represented by this object

Throws:

GenericSignatureFormatError - if the generic class signature does not conform to the format specified in *The Java™ Virtual Machine Specification*

TypeNotPresentException - if the generic superclass refers to a non-existent type declaration

MalformedParameterizedTypeException - if the generic superclass refers to a parameterized type that cannot be instantiated for any reason

Since:

1.5

getPackage

```
public Package getPackage()
```

Gets the package for this class. The class loader of this class is used to find the package. If the class was loaded by the bootstrap class loader the set of packages loaded from CLASSPATH is searched to find the package of the class. Null is returned if no package object was created by the class loader of this class.

Packages have attributes for versions and specifications only if the information was defined in the manifests that accompany the classes, and if

the class loader created the package instance with the attributes from the manifest.

Returns:

`the package of the class, or null if no package information is available from the archive or codebase.`

getInterfaces

```
public Class<?>[] getInterfaces()
```

Determines the interfaces implemented by the class or interface represented by this object.

If this object represents a class, the return value is an array containing objects representing all interfaces implemented by the class. The order of the interface objects in the array corresponds to the order of the interface names in the `implements` clause of the declaration of the class represented by this object. For example, given the declaration:

```
class Shimmer implements FloorWax, DessertTopping
{ ... }
```

suppose the value of `s` is an instance of `Shimmer`; the value of the expression:

```
s.getClass().getInterfaces()[0]
```

is the `Class` object that represents interface `FloorWax`; and the value of:

```
s.getClass().getInterfaces()[1]
```

is the `Class` object that represents interface `DessertTopping`.

If this object represents an interface, the array contains objects representing all interfaces extended by the interface. The order of the interface objects in the array corresponds to the order of the interface names in the `extends` clause of the declaration of the interface represented by this object.

If this object represents a class or interface that implements no interfaces, the method returns an array of length 0.

If this object represents a primitive type or void, the method returns an array of length 0.

If this `Class` object represents an array type, the interfaces `Cloneable` and `java.io.Serializable` are returned in that order.

Returns:

`an array of interfaces implemented by this class.`

getGenericInterfaces

```
public Type[] getGenericInterfaces()
```

Returns the Types representing the interfaces directly implemented by the class or interface represented by this object.

If a superinterface is a parameterized type, the Type object returned for it must accurately reflect the actual type parameters used in the source code. The parameterized type representing each superinterface is created if it had not been created before. See the declaration of ParameterizedType for the semantics of the creation process for parameterized types.

If this object represents a class, the return value is an array containing objects representing all interfaces implemented by the class. The order of the interface objects in the array corresponds to the order of the interface names in the implements clause of the declaration of the class represented by this object. In the case of an array class, the interfaces Cloneable and Serializable are returned in that order.

If this object represents an interface, the array contains objects representing all interfaces directly extended by the interface. The order of the interface objects in the array corresponds to the order of the interface names in the extends clause of the declaration of the interface represented by this object.

If this object represents a class or interface that implements no interfaces, the method returns an array of length 0.

If this object represents a primitive type or void, the method returns an array of length 0.

Returns:

an array of interfaces implemented by this class

Throws:

GenericSignatureFormatError - if the generic class signature does not conform to the format specified in *The Java™ Virtual Machine Specification*

TypeNotPresentException - if any of the generic superinterfaces refers to a non-existent type declaration

MalformedParameterizedTypeException - if any of the generic superinterfaces refer to a parameterized type that cannot be instantiated for any reason

Since:

1.5

getComponentType

```
public Class<?> getComponentType()
```

Returns the `Class` representing the component type of an array. If this class does not represent an array class this method returns null.

Returns:

`the Class representing the component type of this class if this class is an array`

Since:

`JDK1.1`

See Also:

`Array`

getModifiers

```
public int getModifiers()
```

Returns the Java language modifiers for this class or interface, encoded in an integer. The modifiers consist of the Java Virtual Machine's constants for `public`, `protected`, `private`, `final`, `static`, `abstract` and `interface`; they should be decoded using the methods of class `Modifier`.

If the underlying class is an array class, then its `public`, `private` and `protected` modifiers are the same as those of its component type. If this `Class` represents a primitive type or void, its `public` modifier is always `true`, and its `protected` and `private` modifiers are always `false`. If this object represents an array class, a primitive type or void, then its `final` modifier is always `true` and its interface modifier is always `false`. The values of its other modifiers are not determined by this specification.

The modifier encodings are defined in *The Java Virtual Machine Specification*, table 4.1.

Returns:

`the int representing the modifiers for this class`

Since:

`JDK1.1`

See Also:

`Modifier`

getSigners

```
public Object[] getSigners()
```

Gets the signers of this class.

Returns:

the signers of this class, or null if there are no signers. In particular, this method returns null if this object represents a primitive type or void.

Since:

JDK1.1

getEnclosingMethod

```
public Method getEnclosingMethod()
                            throws SecurityException
```

If this `Class` object represents a local or anonymous class within a method, returns a `Method` object representing the immediately enclosing method of the underlying class. Returns `null` otherwise. In particular, this method returns `null` if the underlying class is a local or anonymous class immediately enclosed by a type declaration, instance initializer or static initializer.

Returns:

the immediately enclosing method of the underlying class, if that class is a local or anonymous class; otherwise null.

Throws:

`SecurityException` - If a security manager, *s*, is present and any of the following conditions is met:

- the caller's class loader is not the same as the class loader of the enclosing class and invocation of `s.checkPermission` method with RuntimePermission("accessDeclaredMembers") denies access to the methods within the enclosing class
- the caller's class loader is not the same as or an ancestor of the class loader for the enclosing class and invocation of `s.checkPackageAccess()` denies access to the package of the enclosing class

Since:

1.5

getEnclosingConstructor

```
public Constructor<?> getEnclosingConstructor()
                                            throws SecurityException
```

If this `Class` object represents a local or anonymous class within a constructor, returns a `Constructor` object representing the immediately enclosing constructor of the underlying class. Returns `null` otherwise. In particular, this method returns `null` if the underlying class is a local or anonymous class immediately enclosed by a type declaration, instance initializer or static initializer.

Returns:

the immediately enclosing constructor of the underlying class, if that class is a local or anonymous class; otherwise null.

Throws:

`SecurityException` - If a security manager, *s*, is present and any of the following conditions is met:

- the caller's class loader is not the same as the class loader of the enclosing class and invocation of `s.checkPermission` method with RuntimePermission("accessDeclaredMembers") denies access to the constructors within the enclosing class
- the caller's class loader is not the same as or an ancestor of the class loader for the enclosing class and invocation of `s.checkPackageAccess()` denies access to the package of the enclosing class

Since:

1.5

getDeclaringClass

```
public Class<?> getDeclaringClass()
                             throws SecurityException
```

If the class or interface represented by this `Class` object is a member of another class, returns the `Class` object representing the class in which it was declared. This method returns null if this class or interface is not a member of any other class. If this `Class` object represents an array class, a primitive type, or void,then this method returns null.

Returns:

the declaring class for this class

Throws:

`SecurityException` - If a security manager, *s*, is present and the caller's class loader is not the same as or an ancestor of the class loader for the declaring class and invocation of

s.checkPackageAccess() denies access to the package of the declaring class

Since:

JDK1.1

getEnclosingClass

```
public Class<?> getEnclosingClass()
                            throws SecurityException
```

Returns the immediately enclosing class of the underlying class. If the underlying class is a top level class this method returns `null`.

Returns:

the immediately enclosing class of the underlying class

Throws:

SecurityException - If a security manager, *s*, is present and the caller's class loader is not the same as or an ancestor of the class loader for the enclosing class and invocation of s.checkPackageAccess() denies access to the package of the enclosing class

Since:

1.5

getSimpleName

```
public String getSimpleName()
```

Returns the simple name of the underlying class as given in the source code. Returns an empty string if the underlying class is anonymous.

The simple name of an array is the simple name of the component type with "[]" appended. In particular the simple name of an array whose component type is anonymous is "[]".

Returns:

the simple name of the underlying class

Since:

1.5

getTypeName

```
public String getTypeName()
```

Return an informative string for the name of this type.

Specified by:

`getTypeName` in interface `Type`

Returns:

an informative string for the name of this type

Since:

1.8

getCanonicalName

```
public String getCanonicalName()
```

Returns the canonical name of the underlying class as defined by the Java Language Specification. Returns null if the underlying class does not have a canonical name (i.e., if it is a local or anonymous class or an array whose component type does not have a canonical name).

Returns:

the canonical name of the underlying class if it exists, and null otherwise.

Since:

1.5

isAnonymousClass

```
public boolean isAnonymousClass()
```

Returns `true` if and only if the underlying class is an anonymous class.

Returns:

true if and only if this class is an anonymous class.

Since:

1.5

isLocalClass

```
public boolean isLocalClass()
```

Returns `true` if and only if the underlying class is a local class.

Returns:

true if and only if this class is a local class.

Since:

1.5

isMemberClass

```
public boolean isMemberClass()
```

Returns `true` if and only if the underlying class is a member class.

Returns:

`true if and only if this class is a member class.`

Since:

1.5

getClasses

```
public Class<?>[] getClasses()
```

Returns an array containing `Class` objects representing all the public classes and interfaces that are members of the class represented by this `Class` object. This includes public class and interface members inherited from superclasses and public class and interface members declared by the class. This method returns an array of length 0 if this `Class` object has no public member classes or interfaces. This method also returns an array of length 0 if this `Class` object represents a primitive type, an array class, or void.

Returns:

`the array of Class objects representing the public members of this class`

Throws:

`SecurityException - If a security manager, s, is present and the caller's class loader is not the same as or an ancestor of the class loader for the current class and invocation of s.checkPackageAccess() denies access to the package of this class.`

Since:

JDK1.1

getFields

```
public Field[] getFields()
                  throws SecurityException
```

Returns an array containing `Field` objects reflecting all the accessible public fields of the class or interface represented by this `Class` object.

If this `Class` object represents a class or interface with no no accessible public fields, then this method returns an array of length 0.

If this `Class` object represents a class, then this method returns the public fields of the class and of all its superclasses.

If this `Class` object represents an interface, then this method returns the fields of the interface and of all its superinterfaces.

If this `Class` object represents an array type, a primitive type, or void, then this method returns an array of length 0.

The elements in the returned array are not sorted and are not in any particular order.

Returns:

`the array of Field objects representing the public fields`

Throws:

`SecurityException - If a security manager, s, is present and the caller's class loader is not the same as or an ancestor of the class loader for the current class and invocation of s.checkPackageAccess() denies access to the package of this class.`

Since:

`JDK1.1`

See *The Java™ Language Specification*:

`8.2 Class Members, 8.3 Field Declarations`

getMethods

```
public Method[] getMethods()
                     throws SecurityException
```

Returns an array containing `Method` objects reflecting all the public methods of the class or interface represented by this `Class` object, including those declared by the class or interface and those inherited from superclasses and superinterfaces.

If this `Class` object represents a type that has multiple public methods with the same name and parameter types, but different return types, then the returned array has a `Method` object for each such method.

If this `Class` object represents a type with a class initialization method `<clinit>`, then the returned array does *not* have a corresponding `Method` object.

If this `Class` object represents an array type, then the returned array has a `Method` object for each of the public methods inherited by the array type from `Object`. It does not contain a `Method` object for `clone()`.

If this `Class` object represents an interface then the returned array does not contain any implicitly declared methods from `Object`. Therefore, if no methods are explicitly declared in this interface or any of its superinterfaces then the returned array has length 0. (Note that a `Class` object which represents a class always has public methods, inherited from `Object`.)

If this `Class` object represents a primitive type or void, then the returned array has length 0.

Static methods declared in superinterfaces of the class or interface represented by this `Class` object are not considered members of the class or interface.

The elements in the returned array are not sorted and are not in any particular order.

Returns:

`the array of Method objects representing the public methods of this class`

Throws:

`SecurityException - If a security manager, s, is present and the caller's class loader is not the same as or an ancestor of the class loader for the current class and invocation of s.checkPackageAccess() denies access to the package of this class.`

Since:

`JDK1.1`

See *The Java™ Language Specification*:

`8.2 Class Members, 8.4 Method Declarations`

getConstructors

```
public Constructor<?>[] getConstructors()
                                  throws SecurityException
```

Returns an array containing `Constructor` objects reflecting all the public constructors of the class represented by this `Class` object. An array of length 0 is returned if the class has no public constructors, or if the class is an array class, or if the class reflects a primitive type or void. Note that while this method returns an array of `Constructor<T>` objects (that is an array of constructors from this class), the return type of this method is `Constructor<?>[]` and *not* `Constructor<T>[]` as might be expected. This less informative return type is necessary since after being returned from this method, the array could be modified to hold `Constructor` objects for different classes, which would violate the type guarantees of `Constructor<T>[]`.

Returns:

the array of Constructor objects representing the public constructors of this class

Throws:

SecurityException - If a security manager, *s*, is present and the caller's class loader is not the same as or an ancestor of the class loader for the current class and invocation of s.checkPackageAccess() denies access to the package of this class.

Since:

JDK1.1

getField

```
public Field getField(String name)
               throws NoSuchFieldException,
                      SecurityException
```

Returns a `Field` object that reflects the specified public member field of the class or interface represented by this `Class` object. The `name` parameter is a `String` specifying the simple name of the desired field.

The field to be reflected is determined by the algorithm that follows. Let C be the class or interface represented by this object:

1. If C declares a public field with the name specified, that is the field to be reflected.
2. If no field was found in step 1 above, this algorithm is applied recursively to each direct superinterface of C. The direct superinterfaces are searched in the order they were declared.
3. If no field was found in steps 1 and 2 above, and C has a superclass S, then this algorithm is invoked recursively upon S. If C has no superclass, then a `NoSuchFieldException` is thrown.

If this `Class` object represents an array type, then this method does not find the `length` field of the array type.

Parameters:

name - the field name

Returns:

the Field object of this class specified by name

Throws:

NoSuchFieldException - if a field with the specified name is not found.

NullPointerException - if name is null

SecurityException - If a security manager, *s*, is present and the caller's class loader is not the same as or an ancestor of the

class loader for the current class and invocation of `s.checkPackageAccess()` denies access to the package of this class.

Since:

JDK1.1

See *The Java™ Language Specification*:

8.2 Class Members, 8.3 Field Declarations

getMethod

```
public Method getMethod(String name,
                        Class<?>... parameterTypes)
                 throws NoSuchMethodException,
                        SecurityException
```

Returns a `Method` object that reflects the specified public member method of the class or interface represented by this `Class` object. The `name` parameter is a `String` specifying the simple name of the desired method. The `parameterTypes` parameter is an array of `Class` objects that identify the method's formal parameter types, in declared order. If `parameterTypes` is `null`, it is treated as if it were an empty array.

If the `name` is "`<init>`" or "`<clinit>`" a `NoSuchMethodException` is raised. Otherwise, the method to be reflected is determined by the algorithm that follows. Let C be the class or interface represented by this object:

1. C is searched for a *matching method*, as defined below. If a matching method is found, it is reflected.
2. If no matching method is found by step 1 then:
    1. If C is a class other than `Object`, then this algorithm is invoked recursively on the superclass of C.
    2. If C is the class `Object`, or if C is an interface, then the superinterfaces of C (if any) are searched for a matching method. If any such method is found, it is reflected.

To find a matching method in a class or interface C: If C declares exactly one public method with the specified name and exactly the same formal parameter types, that is the method reflected. If more than one such method is found in C, and one of these methods has a return type that is more specific than any of the others, that method is reflected; otherwise one of the methods is chosen arbitrarily.

Note that there may be more than one matching method in a class because while the Java language forbids a class to declare multiple methods with the same signature but different return types, the Java virtual machine does not. This increased flexibility in the virtual machine can be used to implement various language features. For example, covariant returns can be implemented with bridge methods; the bridge method and the method being overridden would have the same signature but different return types.

If this Class object represents an array type, then this method does not find the clone() method.

Static methods declared in superinterfaces of the class or interface represented by this Class object are not considered members of the class or interface.

Parameters:

name - the name of the method

parameterTypes - the list of parameters

Returns:

the Method object that matches the specified name and parameterTypes

Throws:

NoSuchMethodException - if a matching method is not found or if the name is "<init>"or "<clinit>".

NullPointerException - if name is null

SecurityException - If a security manager, *s*, is present and the caller's class loader is not the same as or an ancestor of the class loader for the current class and invocation of s.checkPackageAccess() denies access to the package of this class.

Since:

JDK1.1

See *The Java™ Language Specification*:

8.2 Class Members, 8.4 Method Declarations

getConstructor

```
public Constructor<T> getConstructor(Class<?>... parameterTypes)
                              throws NoSuchMethodException,
                                     SecurityException
```

Returns a Constructor object that reflects the specified public constructor of the class represented by this Class object. The parameterTypes parameter is an array of Class objects that identify the constructor's formal parameter types, in declared order. If this Class object represents an inner class declared in a non-static context, the formal parameter types include the explicit enclosing instance as the first parameter.

The constructor to reflect is the public constructor of the class represented by this Class object whose formal parameter types match those specified by parameterTypes.

Parameters:

parameterTypes - the parameter array

Returns:

the Constructor object of the public constructor that matches the specified parameterTypes

Throws:

NoSuchMethodException - if a matching method is not found.

SecurityException - If a security manager, *s*, is present and the caller's class loader is not the same as or an ancestor of the class loader for the current class and invocation of s.checkPackageAccess() denies access to the package of this class.

Since:

JDK1.1

getDeclaredClasses

```
public Class<?>[] getDeclaredClasses()
                                throws SecurityException
```

Returns an array of `Class` objects reflecting all the classes and interfaces declared as members of the class represented by this `Class` object. This includes public, protected, default (package) access, and private classes and interfaces declared by the class, but excludes inherited classes and interfaces. This method returns an array of length 0 if the class declares no classes or interfaces as members, or if this `Class` object represents a primitive type, an array class, or void.

Returns:

the array of Class objects representing all the declared members of this class

Throws:

SecurityException - If a security manager, *s*, is present and any of the following conditions is met:

- the caller's class loader is not the same as the class loader of this class and invocation of s.checkPermission method with RuntimePermission("accessDeclaredMembers") denies access to the declared classes within this class
- the caller's class loader is not the same as or an ancestor of the class loader for the current class and invocation of s.checkPackageAccess() denies access to the package of this class

Since:

JDK1.1

getDeclaredFields

```
public Field[] getDeclaredFields()
                            throws SecurityException
```

Returns an array of `Field` objects reflecting all the fields declared by the class or interface represented by this `Class` object. This includes public, protected, default (package) access, and private fields, but excludes inherited fields.

If this `Class` object represents a class or interface with no declared fields, then this method returns an array of length 0.

If this `Class` object represents an array type, a primitive type, or void, then this method returns an array of length 0.

The elements in the returned array are not sorted and are not in any particular order.

Returns:

the array of Field objects representing all the declared fields of this class

Throws:

SecurityException - If a security manager, *s*, is present and any of the following conditions is met:

- the caller's class loader is not the same as the class loader of this class and invocation of `s.checkPermission` method with RuntimePermission("accessDeclaredMembers") denies access to the declared fields within this class
- the caller's class loader is not the same as or an ancestor of the class loader for the current class and invocation of `s.checkPackageAccess()` denies access to the package of this class

Since:

JDK1.1

See *The Java™ Language Specification*:

8.2 Class Members, 8.3 Field Declarations

getDeclaredMethods

```
public Method[] getDeclaredMethods()
                              throws SecurityException
```

Returns an array containing `Method` objects reflecting all the declared methods of the class or interface represented by this `Class` object, including

public, protected, default (package) access, and private methods, but excluding inherited methods.

If this `Class` object represents a type that has multiple declared methods with the same name and parameter types, but different return types, then the returned array has a `Method` object for each such method.

If this `Class` object represents a type that has a class initialization method `<clinit>`, then the returned array does *not* have a corresponding `Method` object.

If this `Class` object represents a class or interface with no declared methods, then the returned array has length 0.

If this `Class` object represents an array type, a primitive type, or void, then the returned array has length 0.

The elements in the returned array are not sorted and are not in any particular order.

Returns:

the array of Method objects representing all the declared methods of this class

Throws:

SecurityException - If a security manager, *s*, is present and any of the following conditions is met:

- the caller's class loader is not the same as the class loader of this class and invocation of s.checkPermission method with RuntimePermission("accessDeclaredMembers") denies access to the declared methods within this class
- the caller's class loader is not the same as or an ancestor of the class loader for the current class and invocation of s.checkPackageAccess() denies access to the package of this class

Since:

JDK1.1

See *The Java™ Language Specification*:

8.2 Class Members, 8.4 Method Declarations

getDeclaredConstructors

```
public Constructor<?>[] getDeclaredConstructors()
                                            throws SecurityException
```

Returns an array of `Constructor` objects reflecting all the constructors declared by the class represented by this `Class` object. These are public, protected, default (package) access, and private constructors. The elements in the array returned are not sorted and are not in any particular order. If the

class has a default constructor, it is included in the returned array. This method returns an array of length 0 if this `Class` object represents an interface, a primitive type, an array class, or void.

See *The Java Language Specification*, section 8.2.

Returns:

the array of Constructor objects representing all the declared constructors of this class

Throws:

`SecurityException` - If a security manager, *s*, is present and any of the following conditions is met:

- the caller's class loader is not the same as the class loader of this class and invocation of `s.checkPermission` method with RuntimePermission("accessDeclaredMembers") denies access to the declared constructors within this class
- the caller's class loader is not the same as or an ancestor of the class loader for the current class and invocation of `s.checkPackageAccess()` denies access to the package of this class

Since:

JDK1.1

getDeclaredField

```
public Field getDeclaredField(String name)
                       throws NoSuchFieldException,
                              SecurityException
```

Returns a `Field` object that reflects the specified declared field of the class or interface represented by this `Class` object. The `name` parameter is a `String` that specifies the simple name of the desired field.

If this `Class` object represents an array type, then this method does not find the `length` field of the array type.

Parameters:

name - the name of the field

Returns:

the Field object for the specified field in this class

Throws:

`NoSuchFieldException` - if a field with the specified name is not found.

`NullPointerException` - if name is null

SecurityException - If a security manager, *s*, is present and any of the following conditions is met:

- the caller's class loader is not the same as the class loader of this class and invocation of s.checkPermission method with RuntimePermission("accessDeclaredMembers") denies access to the declared field
- the caller's class loader is not the same as or an ancestor of the class loader for the current class and invocation of s.checkPackageAccess() denies access to the package of this class

Since:

JDK1.1

See *The Java™ Language Specification*:

8.2 Class Members, 8.3 Field Declarations

getDeclaredMethod

```
public Method getDeclaredMethod(String name,
                                Class<?>... parameterTypes)
                         throws NoSuchMethodException,
                                SecurityException
```

Returns a Method object that reflects the specified declared method of the class or interface represented by this Class object. The name parameter is a String that specifies the simple name of the desired method, and the parameterTypes parameter is an array of Class objects that identify the method's formal parameter types, in declared order. If more than one method with the same parameter types is declared in a class, and one of these methods has a return type that is more specific than any of the others, that method is returned; otherwise one of the methods is chosen arbitrarily. If the name is "<init>"or "<clinit>" a NoSuchMethodException is raised.

If this Class object represents an array type, then this method does not find the clone() method.

Parameters:

name - the name of the method

parameterTypes - the parameter array

Returns:

the Method object for the method of this class matching the specified name and parameters

Throws:

NoSuchMethodException - if a matching method is not found.

NullPointerException - if name is null

SecurityException - If a security manager, *s*, is present and any of the following conditions is met:

- the caller's class loader is not the same as the class loader of this class and invocation of s.checkPermission method with RuntimePermission("accessDeclaredMembers") denies access to the declared method
- the caller's class loader is not the same as or an ancestor of the class loader for the current class and invocation of s.checkPackageAccess() denies access to the package of this class

Since:

JDK1.1

See *The Java™ Language Specification*:

8.2 Class Members, 8.4 Method Declarations

getDeclaredConstructor

```
public Constructor<T> getDeclaredConstructor(Class<?>... parameterTypes)
                                      throws NoSuchMethodException,
                                             SecurityException
```

Returns a Constructor object that reflects the specified constructor of the class or interface represented by this Class object. The parameterTypes parameter is an array of Class objects that identify the constructor's formal parameter types, in declared order. If this Class object represents an inner class declared in a non-static context, the formal parameter types include the explicit enclosing instance as the first parameter.

Parameters:

parameterTypes - the parameter array

Returns:

The Constructor object for the constructor with the specified parameter list

Throws:

NoSuchMethodException - if a matching method is not found.

SecurityException - If a security manager, *s*, is present and any of the following conditions is met:

- the caller's class loader is not the same as the class loader of this class and invocation of s.checkPermission method with RuntimePermission("accessDeclaredMembers") denies access to the declared constructor
- the caller's class loader is not the same as or an ancestor of the class loader for the current class and invocation of s.checkPackageAccess() denies access to the package of this class

Since:

JDK1.1

getResourceAsStream

```
public InputStream getResourceAsStream(String name)
```

Finds a resource with a given name. The rules for searching resources associated with a given class are implemented by the defining class loader of the class. This method delegates to this object's class loader. If this object was loaded by the bootstrap class loader, the method delegates to `ClassLoader.getSystemResourceAsStream(java.lang.String)`.

Before delegation, an absolute resource name is constructed from the given resource name using this algorithm:

- If the `name` begins with a `'/'` (`'\u002f'`), then the absolute name of the resource is the portion of the `name` following the `'/'`.
- Otherwise, the absolute name is of the following form:

  ```
  modified_package_name/name
  ```

  Where the `modified_package_name` is the package name of this object with `'/'` substituted for `'.'` (`'\u002e'`).

Parameters:

`name - name of the desired resource`

Returns:

`A InputStream object or null if no resource with this name is found`

Throws:

`NullPointerException - If name is null`

Since:

JDK1.1

getResource

```
public URL getResource(String name)
```

Finds a resource with a given name. The rules for searching resources associated with a given class are implemented by the defining class loader of the class. This method delegates to this object's class loader. If this object was loaded by the bootstrap class loader, the method delegates to `ClassLoader.getSystemResource(java.lang.String)`.

Before delegation, an absolute resource name is constructed from the given resource name using this algorithm:

- If the name begins with a '/' ('\u002f'), then the absolute name of the resource is the portion of the name following the '/'.
- Otherwise, the absolute name is of the following form:

```
modified_package_name/name
```

  Where the modified_package_name is the package name of this object with '/' substituted for '.' ('\u002e').

Parameters:

name - name of the desired resource

Returns:

A URL object or null if no resource with this name is found

Since:

JDK1.1

getProtectionDomain

```
public ProtectionDomain getProtectionDomain()
```

Returns the ProtectionDomain of this class. If there is a security manager installed, this method first calls the security manager's checkPermission method with a RuntimePermission("getProtectionDomain") permission to ensure it's ok to get the ProtectionDomain.

Returns:

the ProtectionDomain of this class

Throws:

SecurityException - if a security manager exists and its checkPermission method doesn't allow getting the ProtectionDomain.

Since:

1.2

See Also:

ProtectionDomain, SecurityManager.checkPermission(java.security.Permission), RuntimePermission

desiredAssertionStatus

```
public boolean desiredAssertionStatus()
```

Returns the assertion status that would be assigned to this class if it were to be initialized at the time this method is invoked. If this class has had its assertion status set, the most recent setting will be returned; otherwise, if any package default assertion status pertains to this class, the most recent setting for the most specific pertinent package default assertion status is returned; otherwise, if this class is not a system class (i.e., it has a class loader) its class loader's default assertion status is returned; otherwise, the system class default assertion status is returned.

Few programmers will have any need for this method; it is provided for the benefit of the JRE itself. (It allows a class to determine at the time that it is initialized whether assertions should be enabled.) Note that this method is not guaranteed to return the actual assertion status that was (or will be) associated with the specified class when it was (or will be) initialized.

Returns:

`the desired assertion status of the specified class.`

Since:

`1.4`

See Also:

`ClassLoader.setClassAssertionStatus(java.lang.String, boolean)`, `ClassLoader.setPackageAssertionStatus(java.lang.String, boolean)`, `ClassLoader.setDefaultAssertionStatus(boolean)`

isEnum

```
public boolean isEnum()
```

Returns true if and only if this class was declared as an enum in the source code.

Returns:

`true if and only if this class was declared as an enum in the source code`

Since:

`1.5`

getEnumConstants

```
public T[] getEnumConstants()
```

Returns the elements of this enum class or null if this Class object does not represent an enum type.

Returns:

an array containing the values comprising the enum class represented by this Class object in the order they're declared, or null if this Class object does not represent an enum type

Since:

1.5

cast

```
public T cast(Object obj)
```

Casts an object to the class or interface represented by this `Class` object.

Parameters:

`obj` - the object to be cast

Returns:

the object after casting, or null if obj is null

Throws:

`ClassCastException` - if the object is not null and is not assignable to the type T.

Since:

1.5

asSubclass

```
public <U> Class<? extends U> asSubclass(Class<U> clazz)
```

Casts this `Class` object to represent a subclass of the class represented by the specified class object. Checks that the cast is valid, and throws a `ClassCastException` if it is not. If this method succeeds, it always returns a reference to this class object.

This method is useful when a client needs to "narrow" the type of a `Class` object to pass it to an API that restricts the `Class` objects that it is willing to accept. A cast would generate a compile-time warning, as the correctness of the cast could not be checked at runtime (because generic types are implemented by erasure).

Type Parameters:

`U` - the type to cast this class object to

Parameters:

`clazz` - the class of the type to cast this class object to

Returns:

this Class object, cast to represent a subclass of the specified class object.

Throws:

`ClassCastException` - if this Class object does not represent a subclass of the specified class (here "subclass" includes the class itself).

Since:

1.5

getAnnotation

```
public <A extends Annotation> A getAnnotation(Class<A> annotationClass)
```

Description copied from interface: `AnnotatedElement`

Returns this element's annotation for the specified type if such an annotation is *present*, else null.

Specified by:

`getAnnotation` in interface `AnnotatedElement`

Type Parameters:

`A` - the type of the annotation to query for and return if present

Parameters:

`annotationClass` - the Class object corresponding to the annotation type

Returns:

this element's annotation for the specified annotation type if present on this element, else null

Throws:

`NullPointerException` - if the given annotation class is null

Since:

1.5

isAnnotationPresent

```
public boolean isAnnotationPresent(Class<? extends Annotation> annotationClass)
```

Returns true if an annotation for the specified type is *present* on this element, else false. This method is designed primarily for convenient access to marker annotations.

The truth value returned by this method is equivalent to:
`getAnnotation(annotationClass) != null`

The body of the default method is specified to be the code above.

Specified by:

isAnnotationPresent in interface AnnotatedElement

Parameters:

annotationClass - the Class object corresponding to the annotation type

Returns:

true if an annotation for the specified annotation type is present on this element, else false

Throws:

NullPointerException - if the given annotation class is null

Since:

1.5

getAnnotationsByType

```
public <A extends Annotation> A[] getAnnotationsByType(Class<A> annotationClass)
```

Description copied from interface: AnnotatedElement

Returns annotations that are *associated* with this element. If there are no annotations *associated* with this element, the return value is an array of length 0. The difference between this method and AnnotatedElement.getAnnotation(Class) is that this method detects if its argument is a *repeatable annotation type* (JLS 9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

getAnnotationsByType in interface AnnotatedElement

Type Parameters:

A - the type of the annotation to query for and return if present

Parameters:

annotationClass - the Class object corresponding to the annotation type

Returns:

all this element's annotations for the specified annotation type if associated with this element, else an array of length zero

Throws:

NullPointerException - if the given annotation class is null

Since:

1.8

getAnnotations

```
public Annotation[] getAnnotations()
```

Description copied from interface: AnnotatedElement

Returns annotations that are *present* on this element. If there are no annotations *present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

`getAnnotations` in interface `AnnotatedElement`

Returns:

annotations present on this element

Since:

1.5

getDeclaredAnnotation

```
public <A extends Annotation> A getDeclaredAnnotation(Class<A> annotationClass)
```

Description copied from interface: AnnotatedElement

Returns this element's annotation for the specified type if such an annotation is *directly present*, else null. This method ignores inherited annotations. (Returns null if no annotations are directly present on this element.)

Specified by:

`getDeclaredAnnotation` in interface `AnnotatedElement`

Type Parameters:

A - the type of the annotation to query for and return if directly present

Parameters:

annotationClass - the Class object corresponding to the annotation type

Returns:

this element's annotation for the specified annotation type if directly present on this element, else null

Throws:

`NullPointerException` - if the given annotation class is null

Since:

1.8

getDeclaredAnnotationsByType

```
public <A extends Annotation> A[] getDeclaredAnnotationsByType(Class<A> annotatio
```

Description copied from interface: AnnotatedElement

Returns this element's annotation(s) for the specified type if such annotations are either *directly present* or *indirectly present*. This method ignores inherited annotations. If there are no specified annotations directly or indirectly present on this element, the return value is an array of length 0. The difference between this method and `AnnotatedElement.getDeclaredAnnotation(Class)` is that this method detects if its argument is a *repeatable annotation type* (JLS 9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation if one is present. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

`getDeclaredAnnotationsByType` in interface `AnnotatedElement`

Type Parameters:

`A` - the type of the annotation to query for and return if directly or indirectly present

Parameters:

`annotationClass` - the Class object corresponding to the annotation type

Returns:

all this element's annotations for the specified annotation type if directly or indirectly present on this element, else an array of length zero

Throws:

`NullPointerException` - if the given annotation class is null

Since:

1.8

getDeclaredAnnotations

```
public Annotation[] getDeclaredAnnotations()
```

Description copied from interface: AnnotatedElement

Returns annotations that are *directly present* on this element. This method ignores inherited annotations. If there are no annotations *directly present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

`getDeclaredAnnotations` in interface `AnnotatedElement`

Returns:

annotations directly present on this element

Since:

1.5

getAnnotatedSuperclass

```
public AnnotatedType getAnnotatedSuperclass()
```

Returns an `AnnotatedType` object that represents the use of a type to specify the superclass of the entity represented by this `Class` object. (The *use* of type Foo to specify the superclass in '... extends Foo' is distinct from the *declaration* of type Foo.)

If this `Class` object represents a type whose declaration does not explicitly indicate an annotated superclass, then the return value is an `AnnotatedType` object representing an element with no annotations.

If this `Class` represents either the `Object` class, an interface type, an array type, a primitive type, or void, the return value is `null`.

Returns:

an object representing the superclass

Since:

1.8

getAnnotatedInterfaces

```
public AnnotatedType[] getAnnotatedInterfaces()
```

Returns an array of `AnnotatedType` objects that represent the use of types to specify superinterfaces of the entity represented by this `Class` object. (The *use* of type Foo to specify a superinterface in '... implements Foo' is distinct from the *declaration* of type Foo.)

If this `Class` object represents a class, the return value is an array containing objects representing the uses of interface types to specify interfaces implemented by the class. The order of the objects in the array corresponds to the order of the interface types used in the 'implements' clause of the declaration of this `Class` object.

If this `Class` object represents an interface, the return value is an array containing objects representing the uses of interface types to specify interfaces directly extended by the interface. The order of the objects in the

array corresponds to the order of the interface types used in the 'extends' clause of the declaration of this `Class` object.

If this `Class` object represents a class or interface whose declaration does not explicitly indicate any annotated superinterfaces, the return value is an array of length 0.

If this `Class` object represents either the `Object` class, an array type, a primitive type, or void, the return value is an array of length 0.

Returns:

`an array representing the superinterfaces`

Since:

`1.8`

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.